Layered architecture

Restaurant Table Booking & Pre-Ordering App

Name: Kaushik J **SRN: PES1UG24CS223** **SEC: 5D**

For the Restaurant Table Booking & Pre-Ordering App layered architecture was the right solution because the system contains different types of responsibilities that can be clearly separated into different layers. This makes the overall system easier to understand, to test and also to develop.

The architecture is divided into layers presentation layer, business layer and the data layer where each layer has its own responsibility.

The presentation layer is responsible for interaction with the users of the system basically the frontend it contains different interfaces for different users, has diner interface, restaurant manager interface and the kitchen staff interface. These components handle user requests and display the required information without directly dealing with database operations or complex business rules. This separation allows the user interface to be changed without requiring major changes to the underlying business logic.

The Business Layer contains the main functionality of the restaurant system. Components such as the Table & Booking Service, Pre-Order Service, Restaurant Operations Service, Preparation Time Service, and Payment Service are placed here. This layer handles important operations such as checking table availability, making bookings, processing pre-orders, calculating preparation time, and managing table status. Keeping these operations in a separate layer makes the system logic organized and prevents the presentation layer from becoming complicated.

The Data Layer manages the storage and retrieval of information. It contains components such as the Reservation Repository, Order Repository, and Restaurant Database. The business layer communicates with these components instead of directly accessing the database. Therefore, changes to the database structure or storage mechanism can be handled within the data layer without significantly affecting the user interfaces or business services.

layered architecture is separation of concerns. Each layer has a specific responsibility, which reduces unnecessary dependencies between components. The architecture also allows the system to be developed and tested layer by layer. For example, the booking service can be tested independently from the user interface and database implementation.

One test case: when a diner selects an available table, the request starts from the Diner Interface, is handled by the Table & Booking Service, and the required reservation information is obtained through the Reservation Repository and database.

Conclusion: Layered Architecture was selected because it provides clear separation of responsibilities, reduces coupling between different parts of the system, simplifies maintenance

and testing, and provides a structured architecture for implementing the restaurant's booking, pre-ordering, and management functions.